

3

ReSafety: the process

In this chapter, the RESafety process for modeling safety requirements and generating the safety analysis artifact is described. The chapter begins with an overview and the overall BPMN model, outlining each step in a procedural manner. A detailed explanation of the six steps then follows, each accompanied by a step-by-step representation using the corresponding BPMN model, as well as an illustrative example applied to the IIP System. Finally, two proposals for alternative uses of the RESafety approach are presented.

3.1 Process Description

Given the need for analysts to elicit and model safety requirements, this work proposes a process designed to support the early identification and specification of such requirements.

It is important to note that, if necessary, the process also supports the modeling of other early system requirements. This is enabled by the iStar4Safety language, which is conservative in nature—that is, it preserves all constructs of the original iStar 2.0 language while incorporating additional elements specific to safety analysis.

The iterative nature of the process facilitates the re-evaluation of the system whenever new safety requirements are identified. In other words, adding new features or components to the current analysis will alter its context—or at the very least, require a new analysis to identify potential new safety issues introduced by these new elements.

In the following, a sequence of steps designed to systematically integrate principles from STPA into iStar4Safety is presented. The process is also modeled using Business Process Model and Notation (BPMN) to enhance clarity and support comprehension through visual artifacts. The BPMN representation of the RESafety process is likewise provided.

The level of granularity in each phase of the process depends on the analyst's understanding of the system. In this context, granularity refers to the degree of detail with which the requirements and system elements are specified. For instance, requirements can initially be addressed at a higher level of abstraction—such as general safety goals—and later refined into more detailed elements—such as specific control actions or constraints. Beginning the

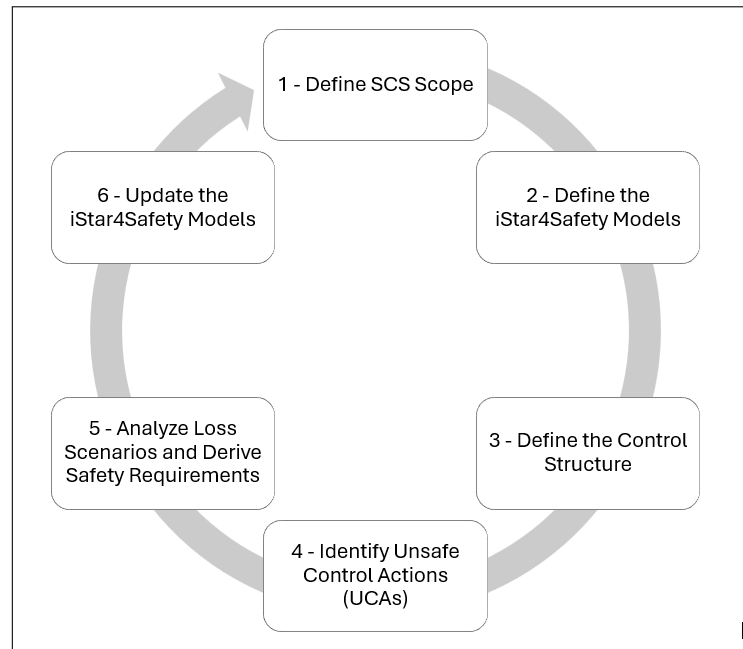
analysis at this higher level may facilitate knowledge management, reducing the risk of omitting essential details when deriving more specific elements from broader concepts (Leveson, 2011). For this purpose, modeling with iStar4Safety—which expresses requirements in the form of actor goals—can be of great assistance. As each iteration contributes to the development of more complete and refined artifacts, the process is characterized as both iterative and incremental.

In this section, which is dedicated to presenting the proposed process, a widely adopted example from the literature—the Insulin Infusion Pump System—is employed. This example was also used in previous studies, including the work that introduced the iStar4Safety modeling language (Ribeiro *et al.*, 2019b; Ribeiro, 2019). The artifact generated during this analysis—specifically, the document detailing the described steps—is available at the following link: *Access the IIPS analysis*.

3.1.1 Iterative Cycle Diagram

The RESafety process comprises six steps, as illustrated in the diagram in Figure 3.1. As shown, the process begins with Step 1: *Define SCS Scope*, and proceeds sequentially to Step 6: *Update the iStar4Safety Models (if needed)*. The circular arrow connecting the last step back to the first indicates that the process may restart, initiating a new iteration. The steps of the RESafety process are:

Figure 3.1: RESafety Iterative Process



Source: Author (2025).

1. Define the SCS Scope
2. Define the iStar4Safety Models

3. Define the Control Structure
4. Identify Unsafe Control Actions (UCAs)
5. Analyze Loss Scenarios and Derive Safety Requirements
6. Update the iStar4Safety Models (if needed)

Step 1 requests elements that are also required in Step 1 of the STPA analysis. Steps 2 and 6 introduce the possibility of modeling the findings using a GORE language—namely, iStar4Safety. The remaining steps, from Step 3 to Step 5, are grounded in the STPA technique and the artifacts generated during the analysis. The following subsection presents an overview of the BPMN model.

3.1.2 RESafety - Process Overview

Figure 3.2 illustrates all the steps of the RESafety process. The analyst—who may be either a requirements engineer or a safety engineer—initiates the process and proceeds through each of the described steps. At the end of Step 6, a decision must be made as to whether a new iteration will be performed or the process should be concluded, in which case it results in a final safety analysis document.

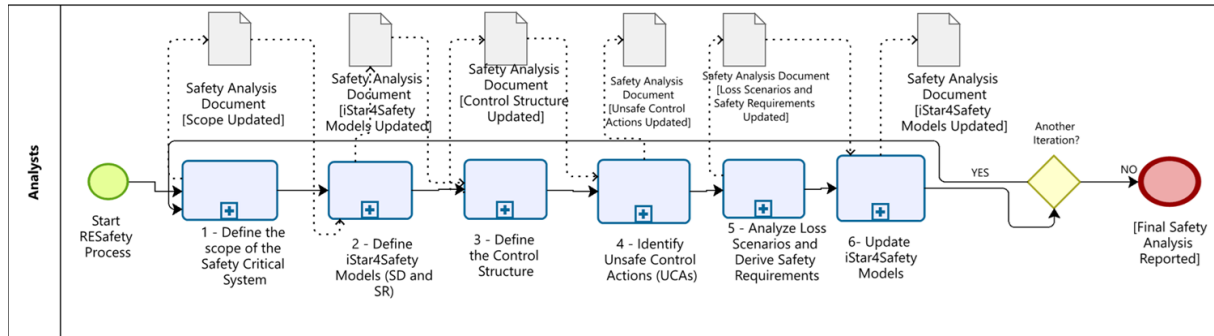
If the process continues to a new iteration, the safety analysis document will be updated at the stage where the iteration resumes. It is important to note, as shown in the BPMN model, that each step generates an updated version of the safety analysis document, reflecting the progress made.

Additionally, the process is designed to be flexible with respect to the order and combination of steps. At the end of this chapter, a section is dedicated to illustrating alternative ways of applying the process, depending on the analyst's decisions.

To access the complete and navigable BPMN model of the RESafety process, visit: ***Access the Complete RESafety BPMN Model.***

Finally, the following subsections provide a detailed explanation of each step, together with an illustrative example based on the IIP System.

Figure 3.2: RESafety Process – Iterative Workflow in BPMN



Source: Author (2025).

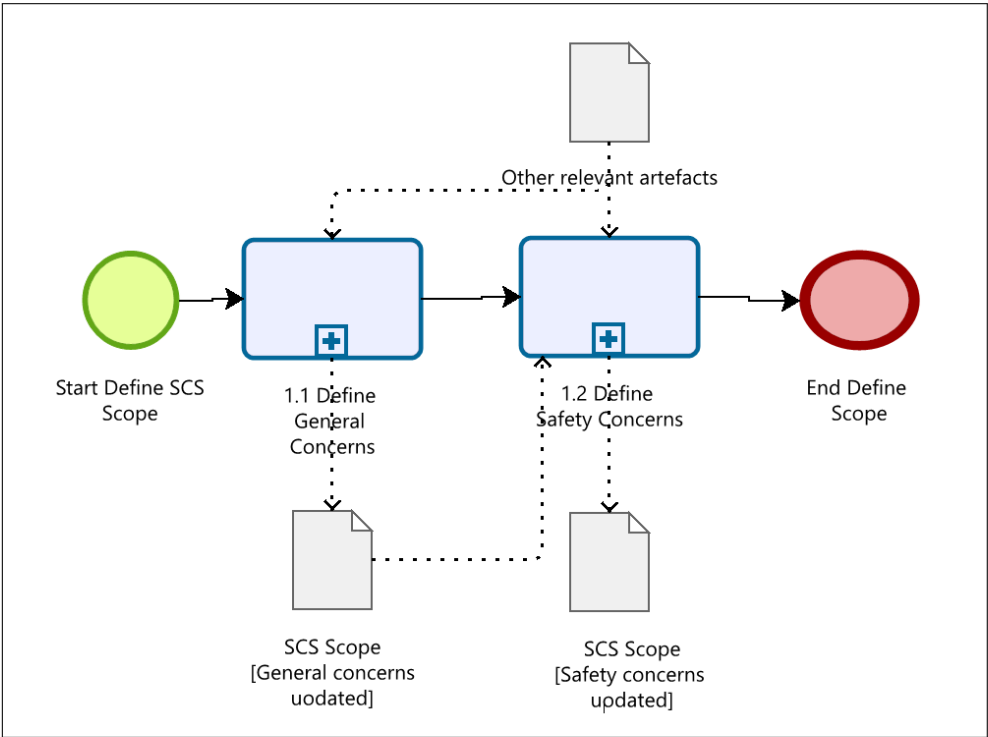
3.1.3 Step 1: Define the scope of the Safety Critical System (SCS)

In the initial phase of the RESafety process, the objective is to define or refine the scope of the system to be modeled. This involves identifying the parts of the system that are relevant for the safety analysis and ensuring that the defined scope aligns with the current stage of development and the available documentation. The output of this phase is an updated Safety Analysis Document [Scope Updated], which serves as the foundation for the subsequent steps.

Given that the process is iterative and incremental, this initial definition may be partial or high-level, particularly during the first iteration. Some elements may be provisionally set or even omitted at this stage. As the process progresses through new cycles, analysts have the opportunity to revisit this scope, refine previously defined aspects, and incorporate new insights or changes arising from the ongoing analysis.

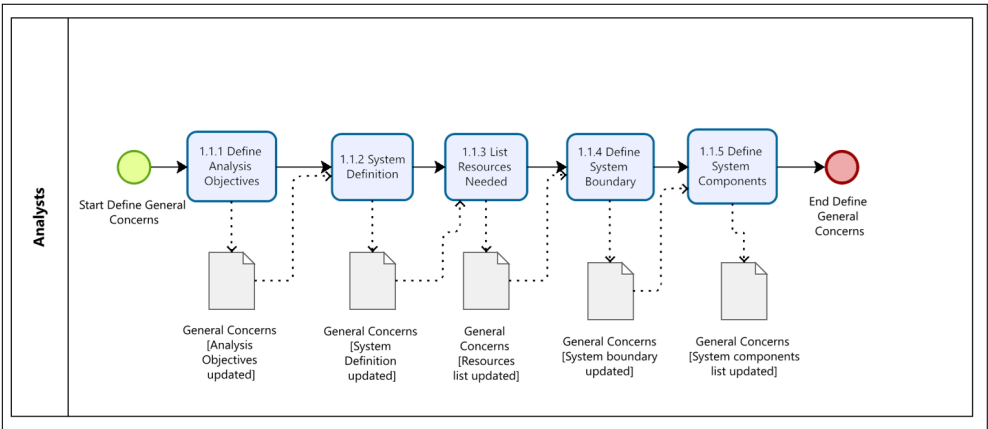
To support this activity, a structured analysis of the elements to be addressed in this step is proposed, categorized into two main groups: General Concerns and Safety Concerns. These categories help ensure a comprehensive and systematic scoping that supports subsequent modeling and analysis efforts. The BPMN model for this step is shown in Figure 3.3, followed by the explanation of each element.

Figure 3.3: Step 1 – Define the scope of the Safety Critical System - BPMN Representation of Step 1 Workflow



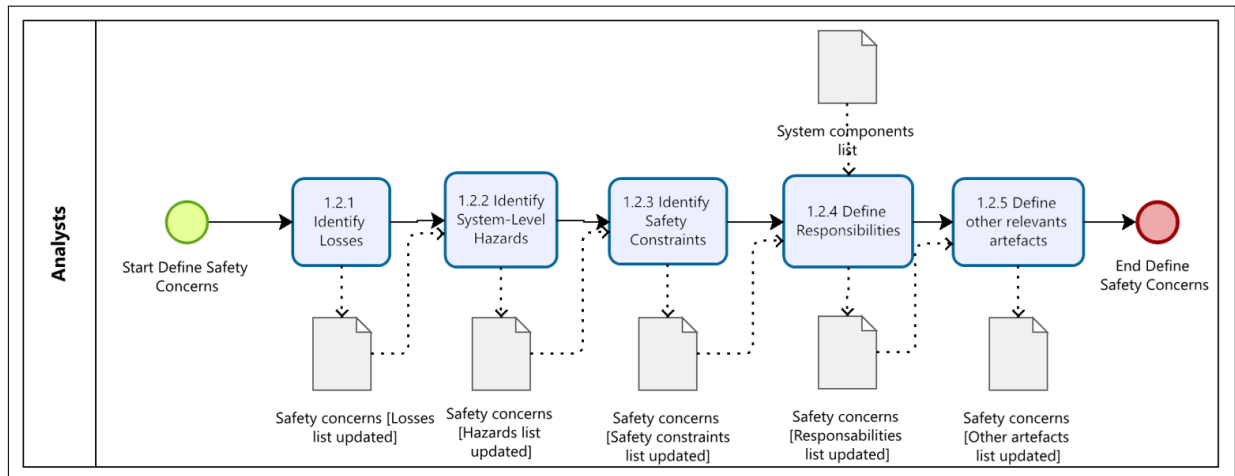
Source: Author (2025).

Figure 3.4: Substep 1.1 – Define general concerns - BPMN Representation of Step 1.1 Workflow



Source: Author (2025).

Figure 3.5: Substep 1.2 – Define safety concerns - BPMN Representation of Step 1.2 Workflow



Source: Author (2025).

- **1.1 - Define General Concerns:** The BPMN model of this subprocess is presented in Figure 3.4.

- **1.1.1 - Define the analysis objectives:** In this activity, the objectives of the analysis must be defined in order to guide the process to be carried out. **IIP System Example:** The purpose of this analysis is to model an Insulin Infusion Pump (IIP) through the iterative RESafety process, generating successive refinements of the system's safety analysis artifacts.
- **1.1.2 - System Definition:** At this stage, the system to be analyzed is briefly introduced, along with a concise explanation of its purpose. **IIP System Example:** The Insulin Infusion Pump (IIP), a safety-critical system, is designed to support the treatment of Type 1 Diabetes Mellitus. Automated IIPs enhance treatment flexibility by managing multiple stages of insulin delivery, effectively mimicking physiological responses. These devices administer both rapid-acting (bolus) and continuous (basal) insulin doses.
- **1.1.3 - List Resources Needed:** At this point, the resources used to conduct the analysis must be listed. These may include system-specific repositories or artifacts, manuals, academic articles, images, prototypes, and other relevant materials. **IIP System Example:** Articles: [Martinazzo \(2022\)](#); [Martins et al. \(2015\)](#); [Zhang et al. \(2011, 2010\)](#); [Bas \(2020\)](#); [Gonzalez Atienza et al. \(2024\)](#), Books [Leveson & Thomas \(2018\)](#); [Martins & Gorschek \(2021\)](#) and other ones, like standards, manuals.
- **1.1.4 - Define the system boundary:** At this stage, the analyst must define the boundaries of the system under analysis. This step is essential for establishing the scope of the analysis, which should ideally focus on the

parts of the system over which control can be exercised to implement safety strategies. **IIP System Example:** The system boundary encompasses activities from the moment the patient configures the infusion settings until the correct dosage is delivered via the catheter.

- 1.1.5 - Define system components: At this stage, the basic components of the system must be identified. These components will be used in system modeling, serving as potential actors in the iStar4Safety models, as well as potential controllers and/or controlled processes in the control structure modeling. **IIP System Example:**
 - Patient
 - Infusion Pump
 - Infusion set

In subsequent iterations, analysts may refine the identification and specification of system components and subcomponents. Within the IIP system, this may include elements such as control buttons, the LCD/audio interface, the microcontroller, the stepper motor and its driver, the mechanical transmission system, and the insulin syringe.

- **1.2 - Define Safety Concerns:** The BPMN model of this subprocess is presented in Figure 3.5.

- **1.2.1 - Identify Losses:** A loss refers to any negative outcome that affects something stakeholders consider valuable. Such outcomes may include fatalities or injuries, damage to property, environmental harm, mission failure, reputational damage, leakage of sensitive information, or any other consequence deemed unacceptable by stakeholders (Leveson & Thomas, 2018). It is important to highlight that, according to Leveson & Thomas (2018), in industry practice the terms losses, accidents, mishaps, and even adverse events are often used interchangeably, which can lead to confusion. In this work, the term loss will be used in order to allow a more generalizable concept. In this approach, losses are labeled using the prefix “Lx”. References to individual components or specific causes, such as “human error”, must be avoided when defining losses (Leveson & Thomas, 2018). **IIP System Example:**
 - L1 - Risk of death
 - L2 - Risk of injury

- **1.2.2 - Identify System-Level Hazards:** Hazards are system states or set of conditions that in the case of a worst-case environmental condition will

lead to losses (Leveson & Thomas, 2018). When identifying hazards, the analyst must specify the associated losses, ensuring traceability between hazards and losses. Furthermore, suppose the analyst already has UCAs - Unsafe Control Actions, identified from a prior STPA analysis or earlier iterations; verifying whether these UCAs are linked with defined hazards is imperative. Otherwise, analysts must specify a new hazard related to the UCA. This is because UCAs are also the causes of hazards. Avoid referencing individual components in the hazard definition. We can use the “Hx” identifier for the hazards. A hazard example, for the loss “L1 - Risk of death” is “H1 - Hypoglycemia [L1, L2]”. **IIP System Example:**

- H1 - Hypoglycemia [L1, L2]
- H2 - Hyperglycemia [L2]

- **1.2.3 - Identify Safety Constraints.** In this step of the process, the safety constraints for the identified hazards must be defined. It is important to note that, at this point, the focus is not on defining specific solutions or implementations for hazard mitigation, but rather on establishing high-level constraints that must not be violated by the system. These constraints can be formulated as simple negations of the hazards and should be traceable to one or more hazards (Leveson & Thomas, 2018). **IIP System Example:**

- SC-01: The system must not administer insulin in excess of the prescribed dose or in unintended circumstances. [H1]
- SC-02: The system must ensure that the prescribed insulin dose is delivered at the correct time and in the correct amount. [H2]

- **1.2.4 - Define the responsibilities:** At this point, the analyst must define the responsibilities of each system actor or component to ensure that the established safety constraints are effectively pursued. It is important to note that the components acting as actors in the iStar4Safety models were identified in Step 1.1.5 – Define System Components. These responsibilities represent refinements of the safety constraints and specify what each entity must do to help enforce them. It is recommended that each component be systematically reviewed against every safety constraint, identifying its specific responsibilities—if any—in ensuring that the constraint is enforced. **IIP System Example:** See table 3.1

- **1.2.5 - Define other relevant artefacts:** Finally, the analyst may define relevant artifacts that can be used or are already available and relate to the system’s safety. **IIP System Example:** User manual of the infusion pump, provided by Medtronic: *Access the manual.*

Table 3.1: Component Responsibilities for IIP System

Component	Responsibility
E1 – Patient (Human Controller)	R-01: Ensure that infusion settings are correctly configured and correspond to the medical prescription [SC-01, SC-02]
E2 – Insulin Pump	R-02: Administer insulin only according to validated infusion parameters and prevent unauthorized dosages [SC-01] R-03: Monitor timing and quantity of delivery to ensure correct dose is given at the right time [SC-02] R-04: Detect anomalies (e.g., occlusions, over-delivery) and alert the user immediately [SC-01, SC-02]
E3 – Infusion Set	R-05: Maintain physical integrity to prevent leaks or unintended flow of insulin [SC-01] R-06: Ensure correct and timely delivery of insulin from pump to patient [SC-02]
E4 – Patient (Human Body)	R-07: Respond physiologically to insulin in a way that is consistent with treatment expectations (acknowledging variability) [SC-02]

Source: Author (2025).

It is important to highlight that the actor Patient was divided into two distinct elements: E1 – Patient (Human Controller) and E4 – Patient (Human Body). This decision was inspired by the work of [Martinazzo \(2022\)](#), which modeled the STPA analysis of a real-world insulin pump project. Therefore, this decomposition appeared plausible to us, as it enables a more precise representation of the roles and interactions involved. Specifically, the human can play an active role as the highest-level process in the control structure, while also fulfilling a passive role as the recipient of the treatment. Finally, documentation, models, safety analyses, and all relevant artifacts associated with the system should be considered as foundational references to support the elicitation of both non-safety and safety requirements. Reusing such materials can help reduce the effort required during the analysis process. These resources may be attached to the analysis itself or referenced and linked to external sources.

3.1.4 Step 2: Define the iStar4Safety Models

In Step 2, RESafety proposes the development of iStar4Safety models. The models — namely, the Strategic Dependency (SD) and the Strategic Rationale (iStar model) / Safety Requirement (RESafety element) (SR) — are used to illustrate the system’s early requirements. This high-level perspective provided by iStar4Safety, through the modeling of requirements based on goals and social relationships among actors, can be seen as a support for an analysis that specifies requirements at a high-enough level, improving the management of the analysis.

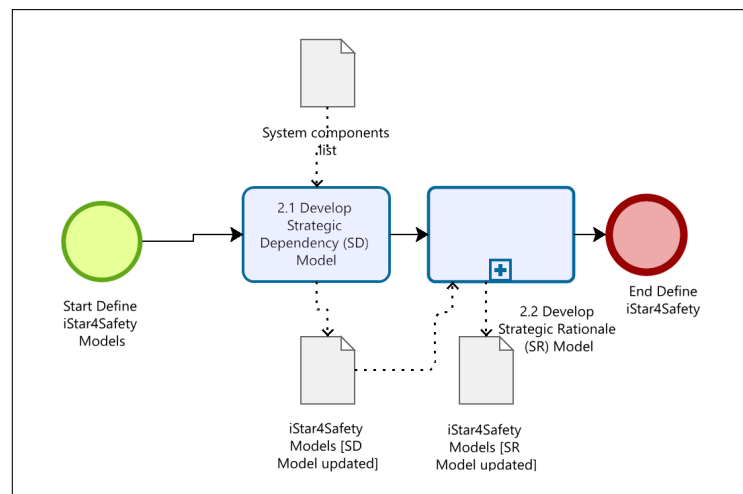
Thus, in Step 2, the actors in the iStar4Safety models will generally correspond to the

components identified in Step 3.1.3 and should be represented accordingly as actors.

Through the SD model, the strategic dependencies among actors are identified, showing who depends on whom and for what (e.g., goals, tasks, resources). This model provides a high-level view of the interactions and responsibilities across system components.

The SR model, in turn, details the internal rationale of each actor, including how they achieve their goals, perform tasks, and manage alternative strategies or conflicting requirements. In this model, the responsibilities defined in Step 1.2.4, as well as the Safety Constraints they aim to address, must be associated with their respective actors. This association supports the justification and traceability of how each safety constraint is considered and enforced within the system's reasoning structure. Figure 3.6 presents the BPM model that illustrates the sequence of steps in this stage.

Figure 3.6: Step 2 - Define iStar4Safety Models (SD and SR) - BPMN Representation of Step 2 Workflow



Source: Author (2025).

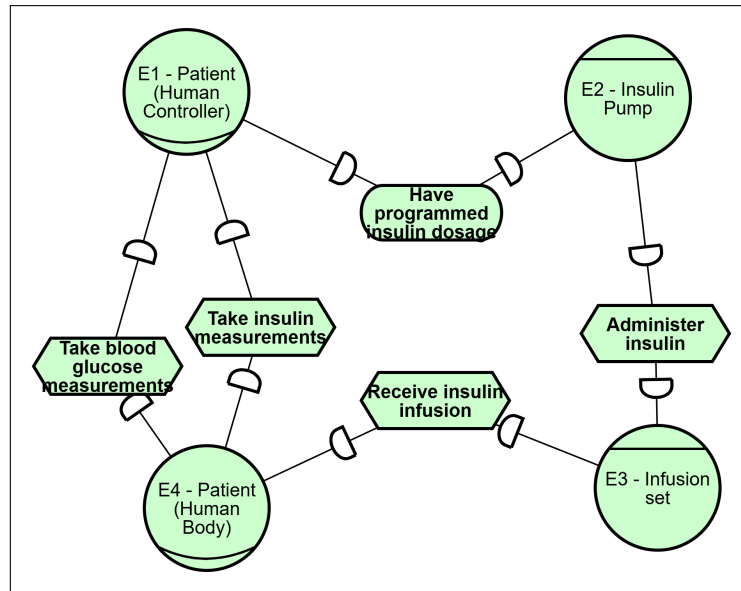
As shown in Figure 3.6, the process begins in Step 2.1 with the development of the SD model and ends in the subprocess of Step 2.2 with the modeling of the SR model. These steps will be further explained below, supported by the IIPS example.

3.1.4.1 SD Model

In Step 2.1 – *Develop Strategic Dependency (SD) Model*, the analyst should have at hand the analysis artifact produced so far—specifically, the list of system components, which is the artifact generated during Step 1.1.5 of Step 1. Based on this, the analyst must analyze and define the actors that will be part of the analysis. Each actor, as well as any element in the analysis, must have an associated ID to support traceability. The actors may be, as defined by iStar4Safety, of type Actor, Agent, or even Role. At this point, for the SD model, the dependencies among the actors should be modeled, which is necessary due to the social interaction nature of iStar4Safety.

It is expected that the actors correspond to those identified in Step 1.1.5. However, the analyst may choose to make relevant adjustments, as will now be illustrated in the IIPS example.

Figure 3.7: IIP System SD Model



Source: Author (2025).

IIP System Example: Figure 3.7 presents the Strategic Dependency (SD) model of iStar4Safety for the IIP System example. Four actors are represented—one more than those defined in Step 1.1.5 of the example. This choice was based on a previous analysis conducted for this pump (Martinazzo, 2022), which was considered an appropriate approach for the early modeling of requirements. The difference lies in the specification of the Patient actor, who may assume both an active role, as a Human Controller – **E1**, and a passive role, representing the human body receiving the treatment – **E4**.

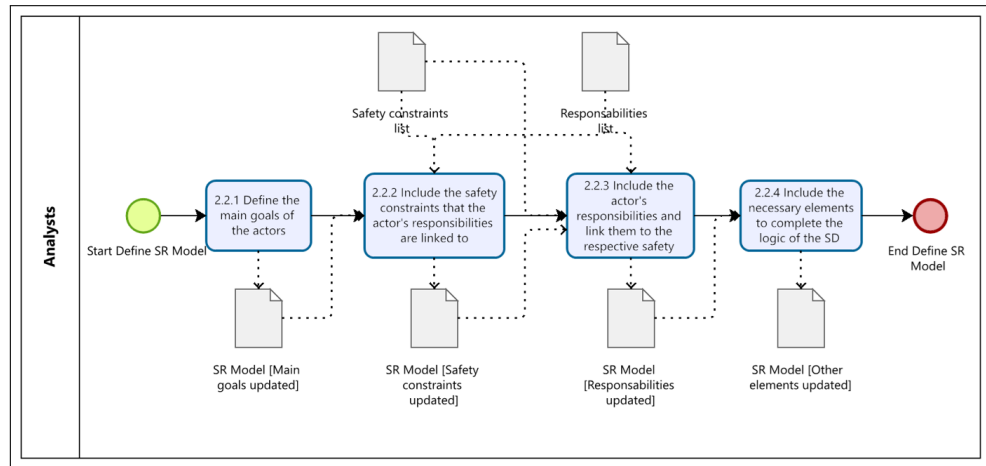
In this model, entity E1 – Patient (Human Controller) depends on entity E2 – Insulin Pump for the "Have programmed insulin dosage" goal. Entity E2 – Insulin Pump, in turn, depends on E3 – Infusion Set to fulfill the "Administer insulin" task. Entity E3 – Infusion Set depends on entity E4 – Patient (Human Body) to perform the "Receive insulin infusion" task. Finally, entity E4 – Patient (Human Body) depends on the patient, now in the role of E1 – Patient (Human Controller), for the "Take blood glucose measurements" and "Take insulin measurements" tasks.

Next, Subprocess 2.2—Develop Strategic Rationale (SR) Model—is described.

3.1.4.2 SR Model

At this stage, the Strategic Rationale (SR) model must be developed. Figure 3.8 depicts the corresponding process in BPM notation.

Figure 3.8: Substep 2.2 – Develop Strategic Rationale (SR) Model -BPMN Representation of Substep 2.2 - Workflow



Source: Author (2025).

In the first iteration, the process begins with the initial SD (Strategic Dependency) model, which, as it already represents the dependencies between actors, may require the inclusion of certain non-safety elements to support this structure.

In Step 2.2.1 – *Define the main goals of the actors*, the overall goal of each actor when interacting with the system must be defined, as the safety logic will be modeled based on this goal. The safety constraints addressed by the actor's responsibilities will be linked to this overall goal.

Therefore, in Step 2.2.2 – *Include the safety constraints that the actor's responsibilities are linked to*, the safety constraints handled by that actor's responsibilities must be modeled as Safety Goals and linked to the actor's main goal. This creates the foundation for modeling all responsibilities associated with that constraint.

In Step 2.2.3 – *Include the actor's responsibilities and link them to the respective safety constraint*, the associated responsibilities defined in Step 1.2.4 of the Safety Concerns are modeled as Safety Goals and connected to the Safety Constraints they are meant to fulfill. These associations must be based on the artifacts *Safety Constraints [Safety Constraints List Updated]* and *Safety Constraints [Responsibilities List Updated]*, generated in Steps 1.2.3 and 1.2.4, respectively.

Finally, in Step 2.2.4 – *Include the necessary elements to complete the logic of the SD*, the analyst may model any remaining dependency elements whose internal logic and fulfillment fall within the scope of that specific actor. After completing all these steps, the SR model is finalized and included in the safety analysis document.

It is important to emphasize that all IDs and associated elements must always be modeled to support traceability among components. All responsibilities and their associated safety constraints should be modeled. The Safety Constraints (SC) element, therefore, should be linked to the actor's overall goal—already modeled—as a Safety Goal. The Responsibilities (R)

associated with that SC must also be connected to it and modeled as Safety Goals.

This modeling approach reflects the rationale that responsibilities are meant to show how each actor contributes to fulfilling the SR, which in turn is a restriction designed to mitigate the occurrence of System-Level Hazards (H) defined in Step 1.2.2. If a R is not performed, the corresponding SR may not be satisfied, which could lead to the occurrence of the associated H modeled in Step 1.2.2—ultimately resulting in the Accidents (A) described in Step 1.2.1.

IIP System Example: The example presented here, shown in Figure 3.9, corresponds to each actor defined in the SD model, now with their boundaries also defined. Due to space constraints, and given that the modeling of the other actors can be understood based on this case, only actor E1 – Patient (Human Controller) is detailed as a representative example.

The main goal of E1 actor is "Use the IIP". Safety constraints SC-01 and SC-02 were added to this goal, since the actor hold responsibility R-01, which aim to address those specific safety constraints. The associations between R-01 and safety constraints SC-01 and SC-02, are indicated at the actor's detailed modeling.

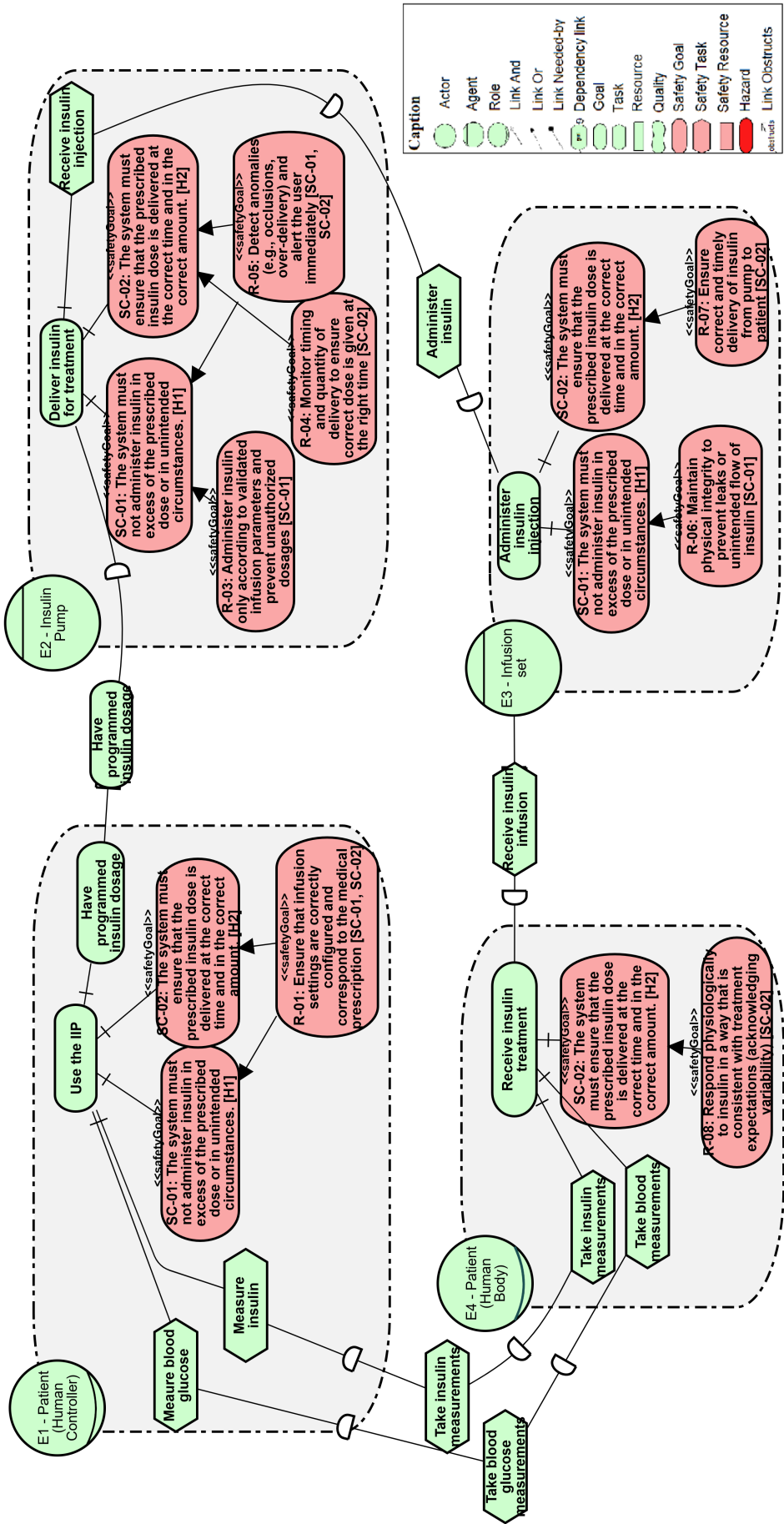
Finally, during the analysis, the need was identified to model additional non-safety elements previously included in the SD model: namely, the goal "Have programmed insulin dosage", fulfilled by actor E2, and the tasks "Measure blood glucose" and "Measure insulin", fulfilled by actor E1 due to its dependency relationship with actor E4.

In the following section, Step 3 is presented, focusing on the modeling of the system's control structure.

3.1.5 Step 3: Define the Control Structure

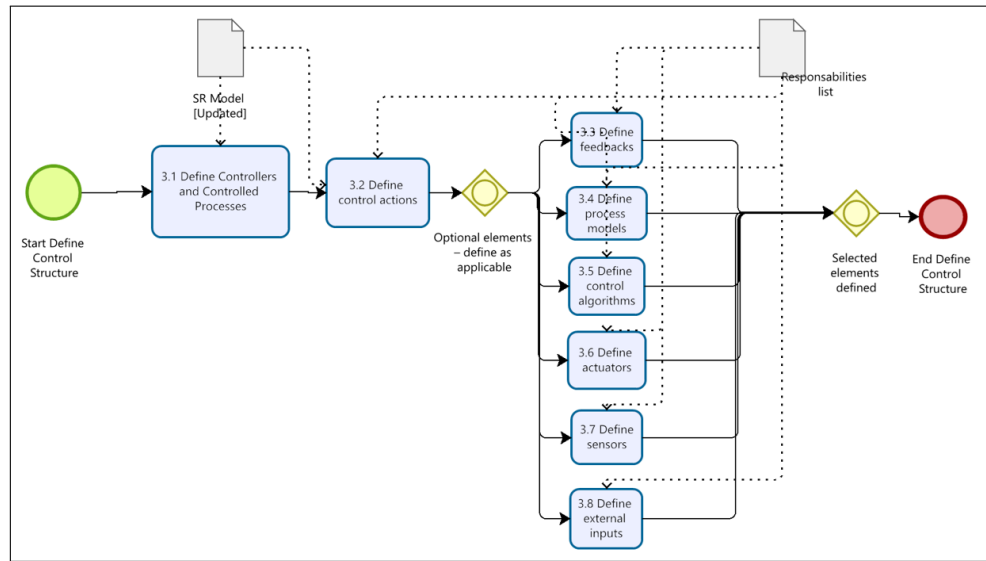
Figure 3.10 presents the BPMN model of Step 3 of the RESafety process, which is dedicated to defining the control structure of the system under analysis. This step is essential to support the identification of UCAs, HCs and safety requirements in subsequent steps. It offers flexibility to the analyst by distinguishing between mandatory and optional modeling elements, depending on the system's complexity and safety relevance.

Figure 3.9: IIP System SR Model.



Source: Author (2025).

Figure 3.10: Step 3 - Define the Control Structure - BPMN Representation of Step 3 Workflow



Source: Author (2025).

The subprocess begins with Step 3.1 – *Define Controllers and Controlled Processes*. In this task, the analyst must define the entities that act as *controllers*—components responsible for decision-making and issuing commands—and those that serve as *controlled processes*—components that react to commands. It is important to note that a single process may simultaneously act as a controller for certain elements while being controlled by others. This definition should be guided by the actors identified in the updated Strategic Rationale (SR) model.

The next step is 3.2 - *Define Control Actions*, where the analyst must identify the control actions issued by the controllers to the controlled processes. These actions are often derived from the responsibilities assigned to each actor in the iStar4Safety model and are fundamental for Step 4, where they are analyzed as potential Unsafe Control Actions (UCAs). This task is mandatory because the process must define control actions to proceed with the UCA analysis.

To model the control actions, the analyst should use the SD model defined in Step 2.1 as a reference. Based on this model, control actions must be derived for each relevant entity. Specifically, in any dependency relationship, the depender (the actor that initiates the dependency) is considered the controller, while the dependee (the actor that fulfills the dependency) is the controlled process. This interpretation stems from the fact that the depender requests something from the dependee; therefore, the dependum of the relationship is modeled as the corresponding control action.

Once these control actions are identified, they should be linked to the responsibilities they support or implement.

An alternative modeling scenario arises when a passive entity is involved—one that does not initiate any control actions and is only influenced by other components. In such cases, this entity should be modeled exclusively as a controlled process. The dependum of its associated

dependency should then be represented as feedback from the controlled process to the controller, reflecting the flow of information rather than a command.

In other words, it is not always possible to associate responsibilities directly with control actions. However, doing so—when applicable—can be beneficial to the analysis. Control actions, when evaluated as potential Unsafe Control Actions (UCAs) in Step 4, enable a more structured and detailed safety assessment than responsibilities addressed solely through other control structure elements.

Nonetheless, the original STPA reference (Leveson & Thomas, 2018) does not require that every responsibility be linked to at least one control action. Responsibilities may be addressed indirectly through components such as feedback loops, process models, or control algorithms, as previously mentioned.

When a responsibility is not directly associated with a control action, it may not lead to the identification of a UCA, and its safety logic might therefore not follow the conventional UCA-to-safety-requirement modeling path. Nonetheless, the absence of an associated control action does not imply that the responsibility is disregarded. In the context of STPA, a hazard is defined as a system state or condition that, when combined with certain environmental or operational circumstances, may lead to a loss (Leveson, 2011). Hazards represent high-level unsafe conditions that can emerge from the behavior of the system as a whole. To complement this perspective, RESafety introduces the concept of Hazardous Conditions (HCs), defined as potential hazardous situations not captured by UCAs but still critical to system safety. HCs provide a mechanism to represent unsafe situations that may arise, for example, from the non-fulfillment of responsibilities or from specific contextual factors. Thus, while hazards constitute the traditional focus of system-theoretic safety analysis, HCs extend the scope by ensuring that relevant unsafe situations not covered by UCAs are also systematically addressed. Analysts are therefore encouraged to introduce HCs when necessary (Ribeiro *et al.*, 2024).

Furthermore, it is important to consider the influence of external inputs—elements outside the system boundary that can still affect its behavior. Although these inputs are not part of the system itself, they may play a crucial role in the fulfillment of responsibilities. For example, a medical prescription issued by a physician (an external actor) directly impacts the responsibility of ensuring that the infusion settings are correctly configured. Therefore, even inputs that fall outside the direct control scope must be properly identified and modeled to support a complete and accurate safety analysis.

Following the two initial steps, the process reaches a **gateway** labeled *Optional elements – define as applicable*. From this point, the analyst may model additional control structure elements based on what is relevant for the system and the depth of understanding already available. The more high-level and abstract the modeling at this stage, is the greater the potential to prevent information loss in later specification phases. However, these optional elements can significantly improve the depth and completeness of the safety analysis.

The optional tasks are as follows:

- **3.3 Define Feedbacks** – Identify and model feedback mechanisms through which controllers receive information about the state of the controlled processes.
- **3.4 Define Process Models** – Specify the internal models used by controllers to understand the process state. These are essential for detecting mismatches between expected and actual behavior.
- **3.5 Define Control Algorithms** – Define the decision-making logic used by controllers to generate control actions. This may involve rules, logic-based routines, or more complex algorithmic behavior.
- **3.6 Define Actuators** – Identify the components responsible for physically or logically implementing the control actions.
- **3.7 Define Sensors** – Define the components that collect process data and transmit it back to controllers, often forming feedback loops.
- **3.8 Define External Inputs** – Model external influences such as user commands, environmental conditions, or other inputs that affect system behavior and safety.

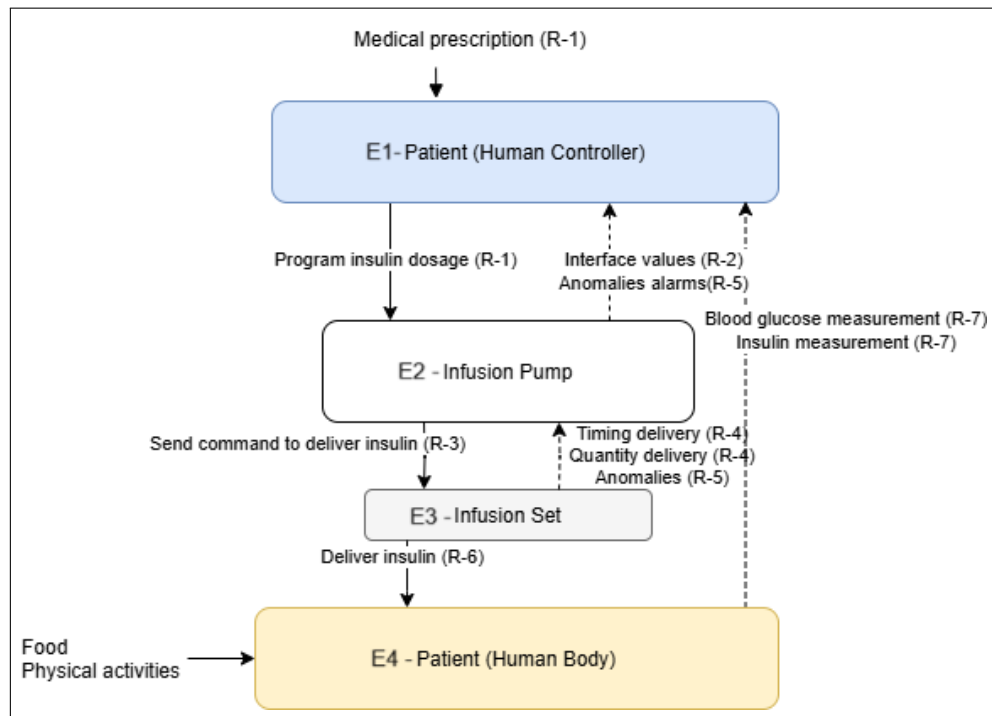
Each of these optional modeling tasks is supported by references to the SR model and the responsibilities list (as indicated by the dashed arrows in the diagram), ensuring consistency and traceability across the analysis.

After the selected elements have been modeled, the subprocess is concluded, resulting in an updated control structure as its main artifact.

It is important to emphasize that this structure makes Step 3 adaptable to the system's design stage and specific project needs. While only Steps 3.1 and 3.2 are mandatory, modeling additional elements can offer greater insight and enhance the robustness of the safety analysis. This flexibility is particularly relevant in early design phases, and the iterative nature of RESafety is intended to address this need by allowing progressive refinement of the control structure throughout successive iterations.

IIP System Example: For the IIP System, considering the previous steps and their inputs, the Control Structure shown in Figure 3.11 was generated. It should be noted that the responsibilities were associated with the elements intended to fulfill them to ensure traceability. We highlight the importance of explicitly associating each responsibility with the elements that address it, in order to support traceability and ensure consistency throughout the modeling process.

Figure 3.11: Control Structure Model of the IIP System



Source: Author (2025).

To model control actions and feedback, the SD model generated in Step 2.1, presented in Figure 3.7, was used as a reference. Table 3.2 presents the associations between elements from the iStar4Safety model and their corresponding elements in the control structure.

Table 3.2: Mapping Between iStar4Safety SD Elements and Control Structure Elements

iStar4Safety SD Element	Representation in iStar4Safety	Control Structure Element	Representation in Control Structure
Have programmed insulin dosage	Goal Dependency – E1 to E2	Program insulin dosage	Control Action – E1 to E2
Administer insulin	Task Dependency – E2 to E3	Send command to deliver insulin	Control Action – E2 to E3
Receive insulin infusion	Task Dependency – E3 to E4	Deliver insulin	Control Action – E3 to E4
Take blood glucose measurement	Task Dependency – E4 to E1	Blood glucose measurement	Feedback – E4 to E1
Take insulin measurement	Task Dependency – E4 to E1	Insulin measurement	Feedback – E4 to E1

Source: Author (2025).

As shown in Figure 3.11, Responsibility R-1 — “Ensure that infusion settings are correctly configured and correspond to the medical prescription [SC-01, SC-02]” — is addressed through the control action “*Program Insulin Dosage (R-1)*”, executed by process E1 over entity E2, and also through the external input “*Medical Prescription (R-1)*”.

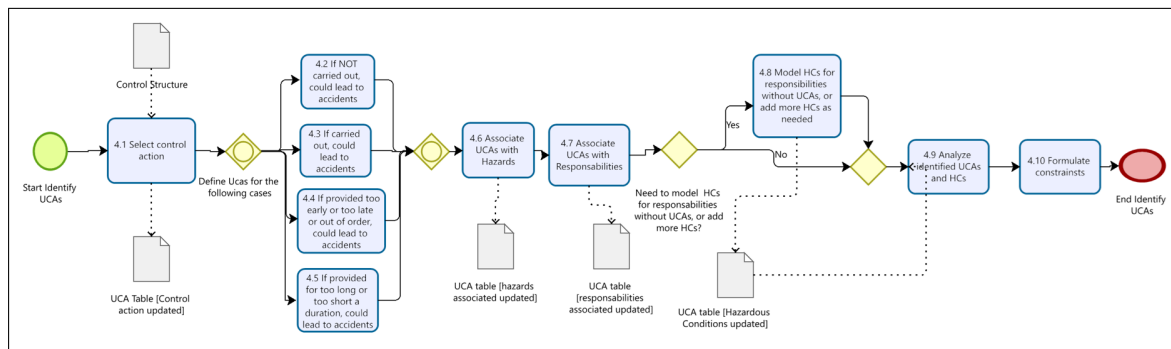
The remaining responsibilities were addressed through control actions and feedback mechanisms accordingly.

It is also important to highlight that the controlled process E4 — *Patient (Human Body)* — due to its passive role in the treatment and inability to perform control actions, had its tasks modeled as feedback. This modeling approach effectively captured the nature of the system and contributed to a coherent representation.

3.1.6 Step 4: Identify Unsafe Control Actions (UCAs)

The BPMN model presented in Figure 3.12 visually represents the subprocess dedicated to identifying Unsafe Control Actions (UCAs), corresponding to Step 4 of the RESafety process. Rooted in the STPA methodology, this step aims to systematically identify control actions that may result in hazardous behavior under specific circumstances. In addition, the identification of Controller Constraints must also be performed, as illustrated in the BPMN model and further detailed in Steps 4.9 and 4.10.

Figure 3.12: Step 4 - Identify Unsafe Control Actions (UCAs) - BPMN Representation of Step 4 Workflow



Source: Author (2025).

With the control structure available, the analyst initiates the subprocess by selecting a control action to be evaluated (Step 4.1). According to Leveson and Thomas (Leveson & Thomas, 2018), four specific situations must be assessed for each control action to determine if it could be unsafe:

- If the control action is **not carried out**, it could lead to losses;
- If the control action is **carried out**, it could lead to losses;
- If the control action is provided **too early, too late, or out of order**, it could lead to losses;
- If the control action is provided for **too long or too short a duration**, it could lead to losses.

Each of these conditions is evaluated individually. When applicable, they give rise to one or more UCAs (Steps 4.2 to 4.5), which are recorded in a UCA table associated with the control action under analysis.

Following this, in Step 4.6, each UCA must be linked to one or more system-level hazards identified earlier in Step 1.1.2. This association is essential for ensuring traceability between potential unsafe behaviors and high-level system hazards. The RESafety approach refines this process by also associating each UCA with the responsibility and corresponding SC that govern its origin (Step 4.7).

To support this traceability, analysts are encouraged to follow this structured reasoning path:

1. Identify the responsibility associated with the Unsafe Control Action (UCA), which should be indicated in the control action that originated it;
2. Determine the Safety Constraint associated with that responsibility;
3. Trace the Safety Constraint to the corresponding system-level Hazard.

This step reinforces the link between behavioral safety issues and the functional responsibilities modeled earlier in the process.

It is important to note that some responsibilities may not result in UCAs, particularly when fulfilled through passive feedback mechanisms or external inputs. As discussed in Section 3.1.5, for instance, Responsibility R-06 — "Ensure correct and timely delivery of insulin from the pump to the patient [SC-02]" — is fulfilled exclusively through feedback (i.e., *Blood glucose measurement (R-6)* and *Insulin measurement (R-6)*) sent from E4 – Patient (Human Body) to E1 – Patient (Human Controller). Since no control action is involved, no UCA is generated. However, the safety impact of this responsibility still needs to be considered.

To address such cases, Step 4.7 includes the task "Associate UCAs with Responsibilities." When a UCA cannot be linked to any responsibility—impacting the completeness of the safety model—RESafety introduces the concept of Hazardous Conditions (HC). These represent potential hazardous situations not captured by UCAs but still critical to system safety. Step 4.8 then allows the analyst to determine whether such HCs should be modeled.

This decision point—"*Need to model HCs for responsibilities without UCAs, or add more HCs?*"—is evaluated under two possible conditions:

- **Yes:** Proceed to Step 4.8, which enables:
 - Modeling HCs for responsibilities not linked to any UCAs due to the absence of associated control actions;
 - Modeling additional hazards not covered by existing UCAs, even if the responsibility already has an associated control action.

- **No:** When no additional HCs are required, the process directly advances to Steps 4.9 and 4.10, where the previously identified UCAs and existing HCs are systematically analyzed to derive the Controller Constraints.

At this point, the analyst transitions from the diagnostic perspective—focused on identifying potential unsafe control actions and hazardous conditions - to the prescriptive perspective, aimed at defining how the controllers must behave to maintain system safety.

Step 4.9: Analyze Identified UCAs and HCs consists of a detailed review of the UCA and HC tables generated in the earlier tasks. In this activity, the analyst examines each unsafe control action and hazardous condition to determine the specific behavioral or operational context that led to its identification. This analysis ensures that the causal relationships between control actions, hazards, and responsibilities are traceable and logically consistent.

Step 4.10: Formulate Controller Constraints operationalizes the safety reasoning developed in previous steps. Each analyzed UCA or HC is translated into one or more explicit Controller Constraints, which describe how the system’s controllers must act—or what they must avoid doing—to prevent unsafe behaviors or hazardous situations. These constraints serve as the formal link between the hazard analysis and the design phase, establishing verifiable safety conditions that can be implemented, monitored, or validated.

In summary, the BPMN model in Figure 3.12 outlines a structured, traceable, and iterative approach for identifying Unsafe Control Actions that integrates system-level hazards, responsibilities, and safety constraints. This approach ensures that:

- All relevant UCAs are systematically identified and evaluated;
- Each UCA is linked to at least one system-level hazard and, when applicable, to one or more responsibilities and safety constraints;
- All responsibilities are addressed—either through UCAs or through associated HC;
- Each UCA and HC results in one or more Controller Constraints, ensuring their translation into actionable safety requirements;
- The safety model evolves through iterative refinement, maintaining consistency across hazards, responsibilities, UCAs, HCs, and constraints.

This integration of behavioral and structural elements within the RESafety process supports a more complete and robust safety assessment.

IIP System Example: For the IIP System now apresentadmos in table 3.3 an example of how a control action can result in each of the four types of Unsafe Control Actions (UCAs). The selected control action is: “Program insulin dosage (R-1)”.

As shown in the table above, it exemplifies the analysis of UCA for a specific control action defined in the control structure: "Program insulin dosage (R-1)". This action is performed

Table 3.3: Unsafe Control Actions (UCAs) for the control action “Program insulin dosage (R-1)”

Control Action	From/To	Not Providing Causes Hazard	Providing Causes Hazard	Too Early, Too Late, Out of Order	Stopped Too Soon, Applied Too Long
Program insulin dosage (R-1)	Patient / Infusion Pump	UCA-01: Patient does not provide “Program insulin dosage” when insulin is required, leading to underdose [H1]	UCA-02: Patient provides “Program insulin dosage” with a value higher than prescribed, leading to overdose [H2] UCA-03: Patient provides “Program insulin dosage” with a value lower than prescribed, leading to underdose [H1]	UCA-04: Patient provides “Program insulin dosage” too late, leading to hyperglycemia [H1] UCA-05: Patient provides “Program insulin dosage” too early, leading to premature insulin administration and resulting in hypoglycemia [H2]	Not applicable

Source: The author (2025).

by E1 – *Patient (Human Controller)* and directed to E2 – *Infusion Pump*. It is associated with Responsibility R-1 and aims to ensure that the insulin dosage delivered by the pump is consistent with the prescribed medical treatment.

For each control action, the RESafety method evaluates its potential to become unsafe under the four STPA-defined conditions. In this case:

- **UCA-01:** If the patient fails to program the infusion pump (i.e., the control action is not provided), the pump will not deliver insulin as needed, potentially leading to **Hazard H1**, which corresponds to the risk of hypoglycemia that may result in serious injury or even death.
- **UCA-02:** If the patient programs an insulin dosage higher than prescribed, this may lead to an overdose and trigger **Hazard H2**.
- **UCA-03:** If the patient programs an insulin dosage lower than prescribed, this may lead to insufficient treatment and trigger **Hazard H1**.
- **UCA-04:** If the patient programs the insulin too late—after the appropriate time window—this may result in hyperglycemia, thus leading to **Hazard H1**.
- **UCA-05:** If the patient programs the insulin too early—before the required time—this may result in hypoglycemia, thus leading to **Hazard H2**.
- The column corresponding to **duration-related issues** (too long or too short) is marked as *Not applicable* in this scenario, indicating that the duration of this control action is not considered a relevant safety concern in this context.

Furthermore, during the decision gateway of the analysis process, the analysts decided to proceed to Step 4.8, as they identified the need to add a new Hazardous Condition (HC)—that is, a potentially dangerous situation not previously covered by the defined UCAs. In this case, the specific condition added was **HC-01: The pump is misplaced or inaccessible to the patient**.

This decision illustrates a key point in the RESafety process: the need to account for responsibilities not addressed by any UCA. In such cases, modeling additional HC ensures comprehensive safety coverage. Additionally, it was determined that in the next iteration, a further analysis would be conducted to assess whether this HC should be linked to an existing system-level hazard or if a new hazard must be defined. This follows the RESafety guideline that every UCA or HC must be associated with at least one hazard. If no appropriate hazard is currently available in the model, a new one must be created.

In the RESafety process, the identification of UCA and HC is directly followed by the definition of the corresponding Controller Constraints. Therefore, the activity previously represented as Step 5 is here integrated into Step 4, maintaining the same logical continuity of analysis.

Once the analyst has completed the identification of UCAs and, when necessary, HCs, the next part of Step 4 focuses on transforming these findings into explicit Controller Constraints. These constraints specify how each controller must behave—or what it must avoid doing—in order to prevent the occurrence of unsafe control actions or hazardous conditions. This integration ensures a seamless transition between the analytical identification of unsafe behavior and the prescriptive definition of safety rules.

In the Step 4 the identification of UCA and HC is directly followed by the definition of the corresponding Controller Constraints. T

By analyzing these artifacts, the analyst derives the specific conditions under which the system must behave to ensure safety. These conditions are then formally translated into constraints during task 4.9- **Formulate Constraints**. Each constraint specifies how a controller must behave—or what it must avoid doing—in order to prevent the associated hazards from materializing.

The result of this substep within Step 4 is a list of **Controller Constraints**, which is updated and stored to support subsequent steps in the RESafety process.

Overall, this integration operationalizes safety by ensuring that every UCA and HC leads to the explicit definition of a constraint that can be implemented or verified in the system's design or runtime behavior. It also ensures traceability by linking each constraint back to the specific hazard scenario or unsafe behavior it addresses.

IIP System Example: Table 3.4 presents the Controller Constraints derived from the analysis of each UCA identified for the IIP System, which has been used as an illustrative example throughout this process. Specifically, the constraints correspond to the UCAs and HC identified for the control action "*Program insulin dosage (R-1)*".

In conclusion, Step 4 now fully encompasses both the identification of Unsafe Control Actions and the definition of the corresponding Controller Constraints. This unified structure provides a continuous analytical and prescriptive flow, ensuring traceability from hazards to responsibilities, UCAs, HCs, and finally to safety-enforcing constraints that can be implemented or verified during system design and operation.

3.1.7 Step 5: Analyze Loss Scenarios and Derive Safety Requirements

In Step 5, the analyst is responsible for examining the Loss Scenario (LS)s, which correspond to the causal factors that lead to the occurrence of UCA and or HC, and consequently, to system hazards. This step can be demanding, given its typically ad hoc modeling nature. In the STPA Handbook (Leveson & Thomas, 2018), the authors outline a set of procedures for identifying such scenarios. Furthermore, a video-based presentation¹ was developed by the authors to introduce a more systematic approach to the identification of loss scenarios.

¹Video titled STPA: Formally Developing Loss Scenarios, available at <https://www.youtube.com/watch?v=hp-KBjIBmrI&t=3936s>

Table 3.4: Controller Constraints for “Program insulin dosage” action

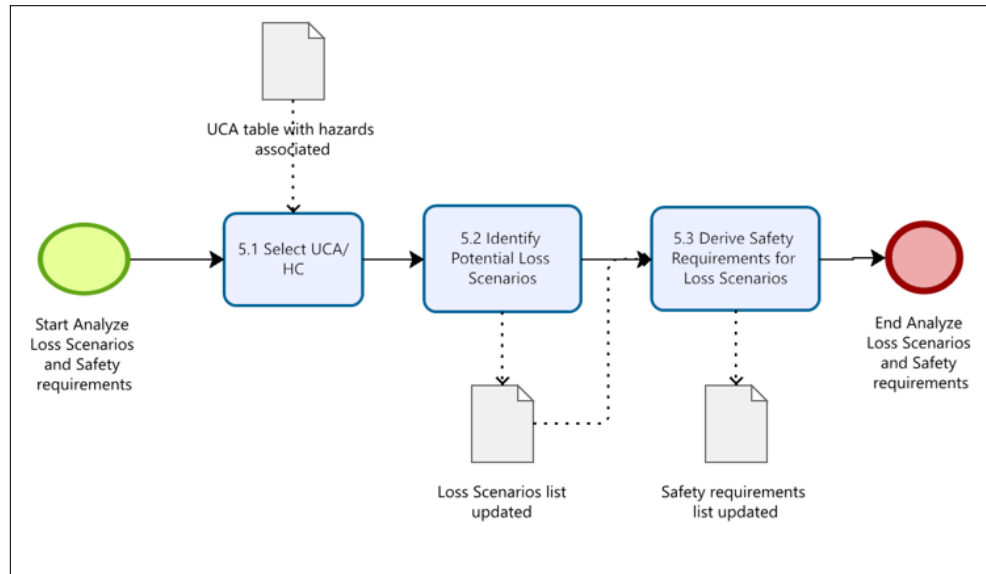
UCA/HC	Controller Constraint
UCA-01: Patient does not provide “Program insulin dosage” when insulin is required, leading to underdose [H1]	C-01: The patient must program the insulin dosage whenever insulin is required, according to clinical guidance. [UCA-01]
UCA-02: Patient provides “Program insulin dosage” with a value higher than prescribed, leading to overdose [H2]	C-02: The patient must ensure the programmed insulin dosage does not exceed the value prescribed by the physician. [UCA-02]
UCA-03: Patient provides “Program insulin dosage” with a value lower than prescribed, leading to underdose [H1]	C-03: The patient must verify that the programmed dosage meets the minimum prescribed threshold to avoid underdosing. [UCA-03]
UCA-04: Patient provides “Program insulin dosage” too late, leading to hyperglycemia [H1]	C-04: The patient must program the insulin dosage in a timely manner, according to the prescribed administration window. [UCA-04]
UCA-05: Patient provides “Program insulin dosage” too early, leading to premature insulin administration and resulting in hypoglycemia [H2]	C-05: The patient must not program the insulin dosage before the appropriate physiological or dietary condition occurs. [UCA-05]
HC-01: The pump is misplaced or inaccessible to the patient.	C-06: The pump must be located in an accessible and known location to the patient at all times. [HC-01]

Source: The author (2025).

Regardless of the method adopted for identifying loss scenarios, the main objective of this step is to uncover them and define corresponding mitigation strategies. These mitigation strategies take the form of safety requirements. To support traceability, all elements—UCAs, loss scenarios, and safety requirements—should be explicitly linked. As highlighted by [Leveson & Thomas \(2018\)](#), the outputs generated during an STPA analysis can serve multiple purposes. These include, but are not limited to, the elicitation of safety requirements, the identification of design recommendations, and the refinement or guidance of the system’s architecture.

Figure 3.13 illustrates, using BPMN notation, the subprocess corresponding to Step 5 of the RESafety process.

Figure 3.13: Step 5 - Analyze loss scenarios and derive safety requirements - BPMN Representation of Step 5 Workflow



Source: Author (2025).

The subprocess begins with the Step **5.1 Select UCA/HC**, where the analyst selects one item at a time from the list of UCAs or HCs (introduced in Step 4). These elements are the entry point for the loss-based reasoning that characterizes this step.

In **Step 5.2 – Identify Potential Loss Scenarios**, the analyst investigates specific combinations of conditions, environmental factors, or operational errors that could cause the selected UCA or HC to result in a system-level hazard. Each potential path leading from the UCA/HC to a hazard is registered as a unique Loss Scenario. These are documented in an evolving list that supports traceability.

Next, in **Step 5.3 – Derive Safety Requirements for Loss Scenarios**, the analyst formulates safety requirements designed to prevent or mitigate the loss scenarios identified. Each SR is derived with the objective of eliminating the causal factors or conditions that would allow the loss to occur. These SRs are recorded in a dedicated artifact and become integral to the overall safety specification of the system.

Throughout this process, traceability is maintained:

- Each Loss Scenario is linked back to one or more UCAs or HCs.
- Each Safety Requirement is explicitly linked to one or more Loss Scenarios.

In summary, Step 5 of the RESafety process supports:

- Exploration of how unsafe behaviors could manifest in practice;
- Formulation of safety requirements tailored to prevent or mitigate those manifestations;

- Traceability between UCAs/HCs, loss scenarios, and associated system-level safety requirements.

IIP System Example: The table 3.5 present the application of Step 5 of the RESafety process to the IIP system. In this step, each identified UCA and HC is analyzed to derive potential LS, which are then used to formulate system-level Safety Requirements (SR).

Table 3.5: Loss Scenarios and Safety Requirements for IIP System

UCA/HC	Loss Scenario (LS)	Safety Requirement (SR)
UCA-01: Patient does not provide "Program insulin dosage" when insulin is required, leading to underdose [H1]	LS-01: The patient forgets to program the dose after the meal, resulting in hyperglycemia. [UCA-01] Martinazzo (2022)	SR-01: The system shall generate an alert if insulin is not programmed within 15 minutes after a meal is detected. [LS-01] Zhang et al. (2011)
	LS-02: The system does not issue a reminder to program the dose after detecting a meal event. [UCA-01] Ribeiro et al. (2024)	SR-02: The interface must maintain a visible warning if no insulin programming is detected post-meal. [LS-02] Ribeiro et al. (2024)
UCA-02: Patient provides "Program insulin dosage" with a value higher than prescribed, leading to overdose [H2]	LS-03: The patient repeats a bolus due to lack of feedback on recent insulin administration. [UCA-02] Zhang et al. (2010)	SR-03: The system shall display recent insulin activity clearly before accepting a new dose. [LS-03] Zhang et al. (2011)
	LS-04: Patient misinterprets the prescribed dose and enters a value higher than medically indicated. [UCA-02] Zhang et al. (2011)	SR-04: The system shall cross-check manual input with prescription data and alert if excess dosage is detected. [LS-04] Zhang et al. (2011)
UCA-03: Patient provides "Program insulin dosage" with a value lower than prescribed, leading to underdose [H1]	LS-05: The patient reduces the dose to avoid hypoglycemia without clinical basis. [UCA-03] Martinazzo (2022)	SR-05: The system must recommend confirmation when the user's dose is significantly below the recommended amount. [LS-05, LS-06] Zhang et al. (2011)
	LS-06: The system does not notify that the entered dose is below clinical expectation. [UCA-03] Zhang et al. (2011)	
UCA-04: Patient provides "Program insulin dosage" too late, leading to hyperglycemia [H1]	LS-07: The patient delays programming due to being busy or distracted, compromising glycemic control. [UCA-04] Martinazzo (2022)	SR-06: The interface must issue periodic prompts for pending bolus if blood glucose remains elevated and no dose is scheduled. [LS-07] Zhang et al. (2011)
	LS-08: The system accepts bolus entry even after blood glucose spike already occurred. [UCA-04] Ribeiro et al. (2024)	SR-07: The system must block bolus entries considered ineffective post-prandial, requiring physician override. [LS-08] Ribeiro et al. (2024)
UCA-05: Patient provides "Program insulin dosage" too early, leading to premature insulin administration and resulting in hypoglycemia [H2]	LS-09: Patient programs insulin and forgets to eat, leading to insulin drop without carbohydrate intake. [UCA-05] Zhang et al. (2010)	SR-08: System must require user confirmation that the meal is occurring before completing bolus delivery. [LS-09] Zhang et al. (2011)
	LS-10: Patient assumes a meal is imminent, but it is delayed due to unforeseen events. [UCA-05] Martins et al. (2015)	SR-09: If a meal confirmation is not received within a set time, bolus must be suspended or canceled automatically. [LS-10] Martins et al. (2015)
HC-01: The pump is misplaced or inaccessible to the patient.	LS-11: The patient is in a critical condition and does not remember where the pump was placed.	SR-10: The pump must have an associated mobile application that allows a "locate pump" function to trigger an audible alarm when activated.

Source: The author (2025).

As shown, UCAs such as "UCA-01: Patient does not provide 'Program insulin dosage' when insulin is required leading to underdose" lead to specific loss scenarios (e.g., LS-01 and LS-02), both associated with the hazard of hypoglycemia (H1). Each loss scenario is documented with its respective source or rationale. For example, *LS-01* considers a situation where the patient forgets to program the dose after a meal, potentially leading to hyperglycemia, and *SR-01* is derived to require an alert if insulin is not programmed within 15 minutes of a meal.

The analysis continues for other UCAs (UCA-02 to UCA-05) and the HC-01, each mapped to plausible hazardous situations and resulting in concrete safety requirements.

Importantly, this example reinforces the traceability between UCAs/HCs, LSs, and SRs, which is a core principle of RESafety. Each requirement generated is explicitly linked back to one or more loss scenarios and their originating unsafe actions or HCs. This ensures justified set of safety requirements, directly addressing potential failure modes identified throughout the safety analysis process.

3.1.8 Step 6: Update the iStar4Safety Models

Finally, in Step 6—the final step of each iteration—the analyst updates the iStar4Safety models to incorporate all the findings obtained throughout the previous steps of the RESafety process.

At this stage, elements such as safety constraints and responsibilities, previously modeled in Step 2 as *Safety Goals*, are revisited. Each responsibility should be linked to the higher-level safety constraint it contributes to fulfilling, thereby reflecting a refinement and operationalization of that constraint within the system model.

Loss scenarios are treated as *hazard causes*—situations or conditions that can lead to the occurrence of UCAs or HCs. Consequently, each loss scenario must be associated with the specific UCA or HC whose occurrence it contributes to, as established during its definition.

Safety requirements, in turn, are represented in the SR model as *Safety Tasks*. Each task must be explicitly linked to the loss scenarios it is designed to mitigate, ensuring traceability between identified hazards and their corresponding mitigation strategies within the model.

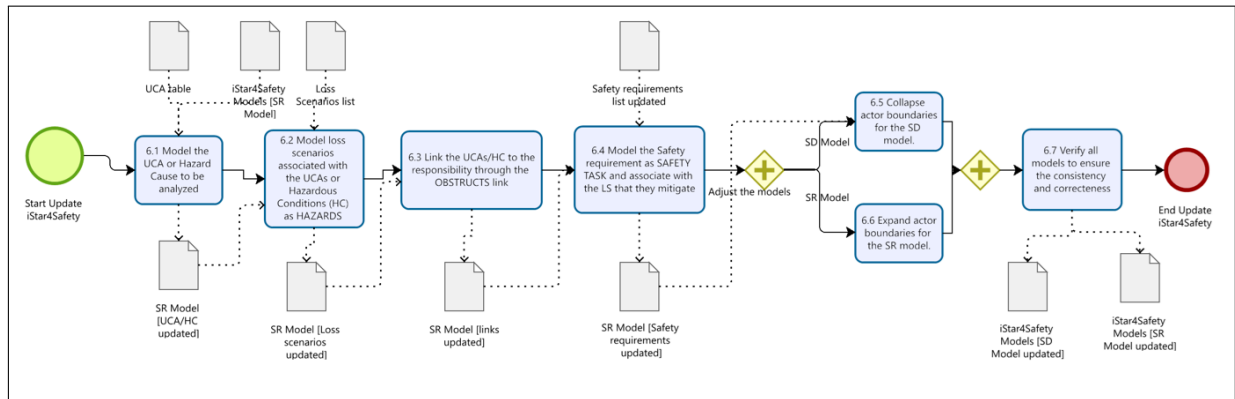
To facilitate traceability, all elements should have clearly defined and consistently applied identifiers. Since iStar4Safety models—particularly the SR model—can become quite large, it is recommended that each element be labeled with the identifier of the corresponding UCA, loss scenario, or safety requirement. Detailed descriptions of these elements should be maintained in the analysis documentation. When using the piStar4Safety tool, these descriptions can be stored in the description field provided for each element in the tool.

The SR model will incorporate the safety logic derived from the RESafety analysis. The SD model may also require adjustments, depending on how the SR model is refined or extended throughout the process.

However, it is important to emphasize that the SD model will not include safety-specific elements. This decision follows the iStar4Safety methodological guideline, which stipulates that for an element to be considered a safety element, it must be accompanied by the full safety reasoning structure: a *Safety Goal*, the *Hazard(s)* that obstruct it, and the corresponding *Mitigation Tasks*. Without this complete logic, the element is not classified as part of the safety analysis.

Figure 3.14 illustrates the activities involved in Step 7 of the RESafety process.

Figure 3.14: Step 7 - Update iStar4Safety Models - BPMN Representation of Step 7 Workflow



Source: Author (2025).

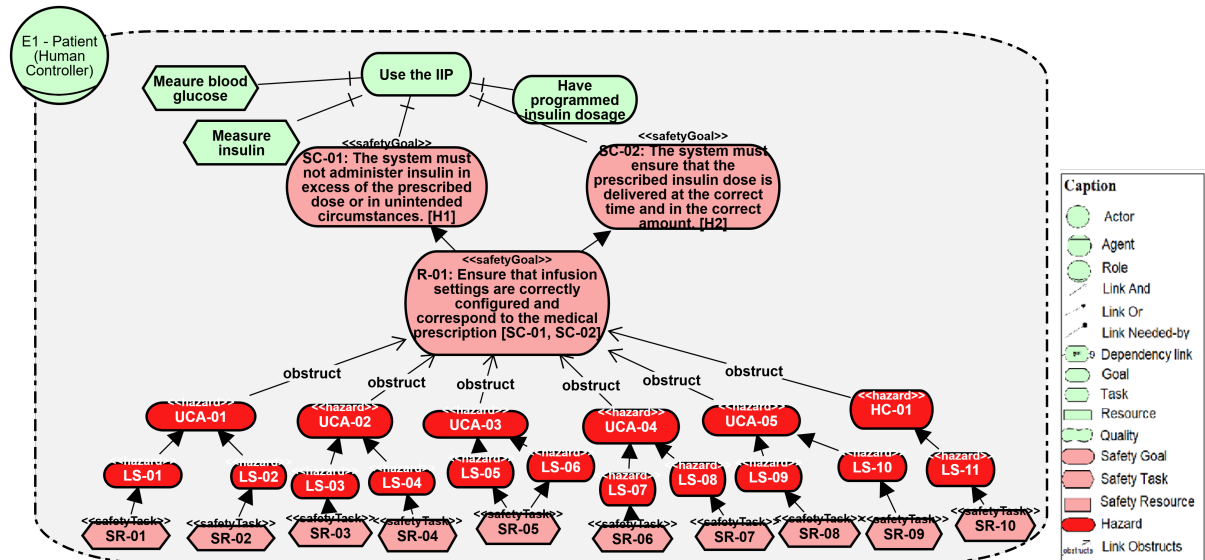
This step is initiated once UCAs and HCs have been analyzed and corresponding Loss Scenarios and Safety Requirements have been identified. The step comprises seven substeps, as detailed below:

- **6.1 Model the UCA or Hazardous Conditions to be analyzed:** The UCA or HC identified in previous steps is included in the SR model of iStar4Safety as an intentional element, safety task, that will be linked to a responsibility.
- **6.2 Model the Loss Scenarios associated with the UCAs or HCs as HAZARDS elements:** Each LS is added as a *Hazard* element within the SR model, maintaining traceability with the originating UCA/HC.
- **6.3 Link the UCAs/HCs to the responsibility through the OBSTRUCTS link:** This step establishes the relationship between the unsafe behavior (UCA/HC) and the system responsibility it endangers, using the "Obstructs" link of the iStar4Safety.
- **6.4 Model the Safety Requirements as SAFETY TASKs:** Each safety requirement derived from the LS analysis is modeled as a *SAFETY TASK*, which mitigates one or more LSs through the OR relationship.
- **6.5 Collapse actor boundaries for the SD model:** For the *Strategic Dependency* (SD) view, actor boundaries are collapsed.
- **6.6 Expand actor boundaries for the SR model:** For the *Strategic Rationale* (SR) view, actor boundaries are expanded to explicitly show internal reasoning elements.
- **6.7 Verify all models to ensure consistency and correctness:** A final review ensures that the SD and SR models are aligned, well-formed, and consistent with the safety analysis outcomes. Artifacts are then updated.

This systematic approach seeks to ensure that all previously identified safety concerns—whether originating from Unsafe Control Actions (UCAs) or Hazardous Conditions (HCs)—are properly reflected and traceable within the iStar4Safety models. It reinforces one of the objective of the RESafety process: to support the modeling of safety requirements while enabling traceability throughout all steps of the analysis.

IIP System Example: Figure 3.15 presents the updated SR model of the insulin infusion pump, which serves as the illustrative example in this chapter. Due to space limitations, only the elements related to the control action “Program insulin dosage” have been modeled. We emphasize that the full diagram will be significantly larger once all control actions, their associated UCAs, loss scenarios, and safety requirements are defined and modeled.

Figure 3.15: Strategic Rationale (SR) model of the IIP System updated in Step 6 of the RESafety process, illustrating the safety logic associated with the control action "Program insulin dosage".



Source: Author (2025).

Here, the reader can observe the layers that compose the initial and complete safety modeling of RESafety: the safety constraints and responsibilities that must be fulfilled to prevent losses; the Unsafe Control Actions (UCAs) and Hazardous Conditions (HCs), which, if they occur, may hinder the fulfillment of those safety constraints and responsibilities, thereby potentially leading to losses; the layer of loss scenarios (LSs), which represent the causes that trigger UCAs and HCs; and finally, the layer of safety requirements (SRs), which completes the safety logic by providing the necessary measures to mitigate the occurrence of such hazards.

3.1.9 Additional Considerations about RESafety

RESafety is a reference process designed to support the early modeling of safety requirements in safety-critical systems. It can be customized by analysts according to the characteristics

of the target system and the available safety-related information. We illustrated this customization in Section 3.2.

The RESafety process is structured into six steps, allowing flexibility in the modeling of responsibilities, safety constraints, unsafe control actions (UCAs), and the derivation of safety requirements. The customization involves selecting relevant elements from the control structure and tailoring the analysis to the specific operational context.

The BPMN diagram describing the RESafety process is available at: <https://monikysribeiro.github.io/resafety.html?lang=en>.

The next section presents a detailed view of how RESafety can be tailored in a real-world context.

3.2 Alternative Uses of the RESafety Process

RESafety is a process designed to support the development of the *Safety Analysis Document*. This artifact includes elements produced throughout each step of the process, namely:

- **STEP 1** – General Concerns and Safety Concerns
- **STEP 2** – Initial iStar4Safety SD and SR models
- **STEP 3** – System Safety Control Structure
- **STEP 4** – Table of Unsafe Control Actions (UCAs) and Hazardous Conditions (HCs) and Table of Controller Constraints
- **STEP 5** – Table of Loss Scenarios and Safety Requirements
- **STEP 6** – Final iStar4Safety SD and SR models of the iteration

Each element can be used as input for the subsequent steps. Therefore, it is important to note that, in some cases, certain elements may already be available. When this occurs, they may be reused, provided they align with the process. It is up to the analyst to determine whether adaptations are needed to ensure compliance with the RESafety process.

In the following subsections, two possible scenarios are presented to illustrate alternative ways of using RESafety. These are provided as suggestions, as the analyst must ultimately assess the specific characteristics of the system and its context to determine how best to apply each step of the process in a feasible and effective manner.

3.2.0.1 iStar Model Already Available

When a system already has an iStar model developed, it can serve as an input for Step 1, particularly for identifying General Concerns. For Step 1.2, which involves defining Safety Concerns, the model should be analyzed to identify elements with safety-critical attributes.

Once Step 1 is completed and the iStar models are available, Step 2 should be carried out using iStar4Safety, as it provides dedicated constructs to model the system's initial safety requirements. From this point, the analysis can proceed through the subsequent steps, according to the desired number of iterations.

3.2.0.2 STPA Analysis Already Available

In cases where a previous STPA analysis exists, the analyst should start by addressing Step 1.1 (General Concerns) and then create the iStar4Safety SD and SR models required for Step 2. This allows the modeling of additional non-safety requirements if desired. It is also important to consider that STPA analyses, especially those following the approach proposed by [Leveson & Thomas \(2018\)](#), typically do not include the modeling of Hazardous Conditions (HCs), which must therefore be developed in Step 4.8.

3.3 Information Management in the RESafety Process

Effective management of the information produced throughout the RESafety process is essential, given the iterative and evolutionary nature of safety analysis. Each step of the process generates specific artifacts, which must be properly organized, versioned, and updated to ensure consistency and traceability. The following guidelines outline how the artifacts of each step can be created, stored, and maintained.

3.3.1 Main Artifact (All Steps): Safety Analysis Document

The main artifact of the RESafety process is the **Safety Analysis Document**, which consolidates all analyses performed, including textual descriptions, tables, and diagrams. Any text editor that supports images and tables can be used for its creation. It is recommended to use collaborative tools such as Google Docs, which facilitate sharing, version control, and team co-editing.

3.3.2 Step 1 – Define the Scope of the SCS

For documenting the system scope, a text editor that allows the inclusion of tables may be employed. Examples include Google Docs or Notion, which provide flexibility and collaborative features suitable for recording the results of this step.

3.3.3 Step 2 – Define iStar4Safety Models (SD and SR)

For modeling the iStar4Safety diagrams, it is recommended to use the **PiStar4Safety** tool, which is open source and specifically designed to support the iStar4Safety language. This tool enforces modeling constraints, preventing invalid relationships and ensuring adherence to

language rules. Alternatively, general-purpose tools such as Draw.io may be used, provided they support all iStar4Safety elements, including the *obstructs* link.

3.3.4 Step 3 – Define the Control Structure

To model the control structure, the Draw.io online tool is recommended, as it provides flexibility for constructing system-theoretic diagrams. The commercial tool **Astah System Safety** can also be used; it natively supports STPA modeling and provides features for defining and visualizing control structures.

3.3.5 Step 4 – Identify Unsafe Control Actions (UCAs)

For this step, a simple text or spreadsheet editor can be used, since the results are presented in tabular format, listing UCAs and, when applicable, HCs. Astah System Safety also offers partial automation for this task, generating UCAs directly from previously defined control actions. Controller constraints can also be created using text or spreadsheet editors, as they are typically organized in tabular form.

3.3.6 Step 5 – Derive Safety Requirements

As in Step 4, a text or spreadsheet editor can be used to document UCAs and HCs associated with their respective loss scenarios and safety requirements.

3.3.7 Step 6 – Update iStar4Safety Models

The same modeling tools used in Step 2 should be applied here, since the SD and SR diagrams are updated to incorporate safety requirements and refined dependencies.

Versioning and Configuration Control

Regarding version control, the use of distributed configuration management tools such as Git is recommended to maintain separate versions for each iteration of the analysis. Proper versioning ensures that the evolution of the RESafety artifacts is systematically recorded and traceable. Each iteration should be clearly labeled to indicate its corresponding version and stage in the process. Maintaining this structure helps preserve key analysis versions, facilitates collaborative work, and supports the iterative refinement that characterizes the RESafety process.

3.3.8 Final Remarks

This information management proposal was suggested by the author as a practical guideline for maintaining consistency and accessibility of all artifacts generated during the

RESafety process. It is important to note that the suggested tools can be adapted according to team preferences and institutional constraints. As future work, the development of an integrated tool is proposed, capable of encompassing all RESafety steps and automating or semi-automating the creation and maintenance of its artifacts.

3.4 Chapter Summary

This chapter presented the RESafety process, composed of six steps that integrate elements from STPA and iStar4Safety. The steps were explained through a BPMN-based model and illustrated with the Insulin Infusion Pump System, a widely adopted example in the literature.

In the next chapter, the evaluations conducted in this research are presented. The first consists of illustrating the process through the analysis of a real system—a medication delivery system that employs a robotic arm in a hospital context. The second comprises an expert-based evaluation involving safety engineering and requirements engineering specialists, carried out using the TAM (Technology Acceptance Model) framework.